

What is Cluster?

A **cluster** is a group of interconnected computers, called **nodes**, that work together as a single unified system. These nodes are connected through high-speed networks and collectively handle tasks to improve **performance, availability, scalability, and reliability**

Examples of cluster applications:

- High-Performance Computing: Used for scientific research, simulations, and other computationally intensive tasks.
- Web Hosting: Clusters can handle high traffic and provide reliable service for websites and web applications

Types Of Cluster

1. Open Cluster: All computers need IP addresses in the Open cluster. These IPs are available on the internet which is a security concern.
2. Close Cluster: All the nodes hide behind the gateway node. This gateway node increases the protection of other nodes.



What is Cluster Computing?

Cluster computing is a model where **multiple computers (nodes)** are connected and work together as a **single unified system**. These nodes collaborate to perform tasks faster, handle larger workloads, and provide higher reliability than a standalone machine.

Each node runs its own OS, is connected via a high-speed LAN or WAN, and shares tasks to increase performance, scalability, and availability.

How does Cluster Computing Works:

- Cluster setup consist of computing nodes, Leader node, Load balancer and Heartbeat Mechanism
- When the leader node receives a request, it passes the task to the computing nodes.

1. Computing Nodes:

Computing nodes are the individual computers inside a cluster that actually perform the processing work. Each node has its own CPU, memory, storage, and OS, and works on a portion of the overall task assigned by the cluster. It is either Homogenous(same hardware) or Heterogenous (diff. hardware).

2. Leader Node:

A leader node is the coordinator of the cluster that receives incoming tasks and assigns work to the other nodes. If it fails, another node is elected to take over so the system keeps running.

3. Load balancer:

A load balancer is a component that distributes incoming requests evenly across multiple nodes in a cluster. It prevents any single node from getting overloaded and keeps the system running efficiently.

4. Heartbeat Mechanism:

The heartbeat mechanism monitors all computing nodes in the cluster to ensure they are operational. When a node fails to respond, the heartbeat mechanism alerts the leader node, redistributing the task to other functional nodes.



Pros:

- **High performance:** Multiple nodes work together, giving much faster processing than a single machine.
- **High availability:** If one node fails, others take over automatically.
- **Scalable:** You can add more nodes anytime to increase capacity.
- **Cost-effective:** Cheaper than buying a supercomputer or mainframe.
- **Flexible:** Nodes can be upgraded or replaced without stopping the entire system.

Cons:

- **Difficult to detect faults:** With many nodes, finding the faulty one can be tricky.
- **Higher hardware cost:** Requires multiple machines, networking equipment, and switches.
- **More space and power needed:** Additional nodes mean more infrastructure.
- **Complex configuration:** Setting up and managing a cluster requires expertise.



Types of Cluster Computing:

1. High-Performance (HP) Cluster

- Designed to solve large scientific or engineering problems using parallel processing.
- Multiple nodes work together on different parts of the same complex task.
- Focuses on speed, computation power, and efficiency.
- Used in simulations, weather forecasting, physics modeling, and research.
- Relies heavily on fast inter-node communication for performance.

2. Load-Balancing Cluster

- Distributes incoming requests evenly across multiple nodes.
- Ensures no single node becomes overloaded or slow.
- Improves system responsiveness during high traffic.
- Commonly used in web hosting, cloud services, and content delivery.

- Uses algorithms like Round Robin or Least Connections.

3. High-Availability (HA) Cluster

- Designed to keep services running even during hardware or software failures.
- Uses redundant nodes that automatically take over if one node fails (failover).
- Ensures minimal downtime and continuous service.
- Common in banking, healthcare, and critical enterprise applications.
- Focuses on reliability, fault tolerance, and data consistency.



Cluster Computing Architecture:

Cluster computing architecture is organized in layers that handle applications, programming models, middleware, and hardware. These layers work together to make multiple computers behave like one powerful system. The architecture ensures performance, fault tolerance, scalability, and smooth communication between nodes.

Components of Cluster Architecture:

1. Applications Layer

- Includes **sequential** and **parallel** applications.
- Parallel applications split tasks across nodes for faster execution.
- This is where user programs interact with the cluster system.

2. Parallel Programming Environment

- Provides tools/libraries to write parallel code.
- Examples: **MPI**, **OpenMP**, **CUDA**, **OpenCL**.

- Helps programmers manage communication and data sharing across nodes.

3. Cluster Middleware

- The “brain” sitting between software and hardware.
- Handles **resource allocation**, **job scheduling**, **fault tolerance**, and **inter-process communication**
- Creates a **Single System Image (SSI)** so the entire cluster looks like one machine.

4. Hardware Layer

- Consists of multiple **workstations/servers (nodes)**.
- Connected using **high-speed networks** (Ethernet, InfiniBand).
- Includes network switches, storage systems, and NICs.
- Hardware determines the raw performance of the cluster.



Cluster Computing Load Balancing Algorithm:

1. Round Robin Algorithm:

Round Robin is a load-balancing algorithm that distributes incoming requests to servers in a fixed circular order. Each new request is sent to the next server in the list, ensuring all nodes get an equal share of the workload. When the end of the list is reached, it loops back to the first server again.

2. Least Connections Algorithm:

The Least Connections algorithm sends a new request to the server that currently has the **fewest active connections**. This keeps the workload balanced by giving more requests to

less-busy nodes, making it ideal when requests vary in size or duration.

3. Beowulf Cluster Algorithm:

A Beowulf cluster is a high-performance computing setup built using **ordinary, commodity computers** connected through a local network. These machines run open-source software and work together to perform parallel processing, giving supercomputer-level performance at low cost.



Key Factors in Clustering Architectures:

- **Scalability:** The cluster should grow by adding more nodes without losing performance.
- **Performance:** Depends on node hardware, network speed, and how efficiently tasks are processed.
- **Fault Tolerance & High Availability:** System must keep running even if a node fails, using redundancy and monitoring.
- **Load Balancing:** Workload should be evenly distributed across nodes to avoid bottlenecks.
- **Network Architecture:** The way nodes connect (Star, Mesh, Ring, Torus, Hypercube) affects communication speed and reliability.



BLAS(Basic Linear Algebra Subprograms):

- **BLAS (Basic Linear Algebra Subprograms)** is a collection of optimized routines used for basic vector and matrix operations in HPC.
- In cluster computing, BLAS routines are used across multiple nodes to accelerate large-scale linear algebra tasks.
- BLAS is divided into three levels:
 - **Level 1:** Vector–vector operations (dot product, scaling, vector addition).
 - **Level 2:** Matrix–vector operations (e.g., matrix–vector multiplication).

- **Level 3:** Matrix–matrix operations (e.g., matrix multiplication), offering the highest performance.

Importance of BLAS in HPC

- **High Performance:** BLAS routines are heavily optimized for CPU/GPU hardware, giving much faster execution than normal linear algebra code.
- **Portable:** Uses a standard interface, so different BLAS libraries can be swapped without changing the actual program.
- **Foundation Layer:** Forms the base for advanced libraries like LAPACK and is used widely in scientific tools.
- **Popular Libraries:** OpenBLAS, Intel MKL, cuBLAS.
- **Used In:** MATLAB, NumPy, and other numerical computing environments.



LAPLACK:

- **LAPACK (Linear Algebra PACKage)** is a widely used numerical library that provides optimized routines for solving linear equations, eigenvalue problems, and singular value decomposition.
- It is designed mainly for **single-node, shared-memory systems**, but in distributed cluster environments it is extended through **ScaLAPACK** for multi-node execution.

Key aspects of LAPLACK in HPC:

- **High Performance:** Utilizes multi-core CPUs and GPUs to handle large matrix computations efficiently.
- **Portable:** Works across major operating systems and hardware platforms, making it suitable for diverse HPC environments.
- **Extensive Functionality:** LAPACK provides a wide range of routines for solving linear systems, least squares problems, eigenvalue and singular value decomposition, and more.

Mission-Critical vs Business-Critical Applications

Mission-Critical Applications	Business-Critical Applications
Required for immediate system operation.	Important but not needed for immediate functioning.
Failure causes complete shutdown or safety risks.	Failure causes financial loss or delay, not total shutdown.
Zero or near-zero downtime allowed.	Short, planned downtime is acceptable.
Needs strong redundancy, failover, and real-time monitoring.	Focuses on continuity, efficiency, and recovery.
Extremely strict performance and reliability requirements.	Reliability needed but not real-time strict.
Used in banking core transactions, ICU monitoring, air-traffic control.	Used in CRM, HR, scheduling, reporting systems.
Failure has severe operational or safety consequences.	Failure affects productivity and revenue but not safety.

Fault Detection

Fault detection is the process of **identifying when a node, process, or component in a distributed system fails or behaves incorrectly**.

It keeps the cluster reliable by spotting failures early so recovery actions can start.

Key techniques

- **Heartbeat mechanism:** Nodes send periodic “alive” signals; missing signals indicate failure.
- **Timeout-based detection:** If a response isn’t received in time, the node is flagged as failed.

- **Failure suspects:** Detect possible crashes based on assumptions and network delays.
 - **Watchdog timers:** Reset or restart unresponsive components automatically.
 - **Checkpointing:** Saves system state so failures can be detected and rolled back safely.
-



Fault Masking

Fault masking ensures the system **continues operating correctly even when faults occur**, hiding the fault from the user.

Instead of just detecting errors, the system corrects or neutralizes them.

Key techniques

- **Triple Modular Redundancy (TMR):** Three modules compute the same output; majority vote decides correct result.
 - **N-Modular Redundancy:** Larger version of TMR for masking multiple failures.
 - **Error-correcting codes (ECC):** Detect and correct bit errors automatically.
 - **Consensus algorithms (Paxos, Raft):** Let the system keep running even if some nodes fail.
 - **Bulkhead isolation:** Faults in one component don't impact others.
-



Checkpointing

Checkpointing is the process of saving the current state of a running application or system so it can resume from the last saved point after a failure, instead of restarting the entire computation.

It reduces computation loss and enables reliable recovery in long-running or mission-critical tasks.

Why Checkpointing is Needed

- Minimizes loss of computation
- Enables recovery after crashes
- Essential for long HPC jobs and distributed systems

Types of Checkpointing

- **Full Checkpointing:** Saves the complete system/app state (memory, files, variables).
- **Incremental Checkpointing:** Saves only the changes since the previous checkpoint.
- **Coordinated Checkpointing:** All processes checkpoint together to maintain global consistency.
- **Uncoordinated Checkpointing:** Each process checkpoints independently, may lead to the *domino effect* (cascading rollbacks).
- **Application-Level Checkpointing:** Implemented within the application for optimized control.
- **System-Level Checkpointing:** Transparent to the application, handled by the OS (e.g., CRIU in Linux).



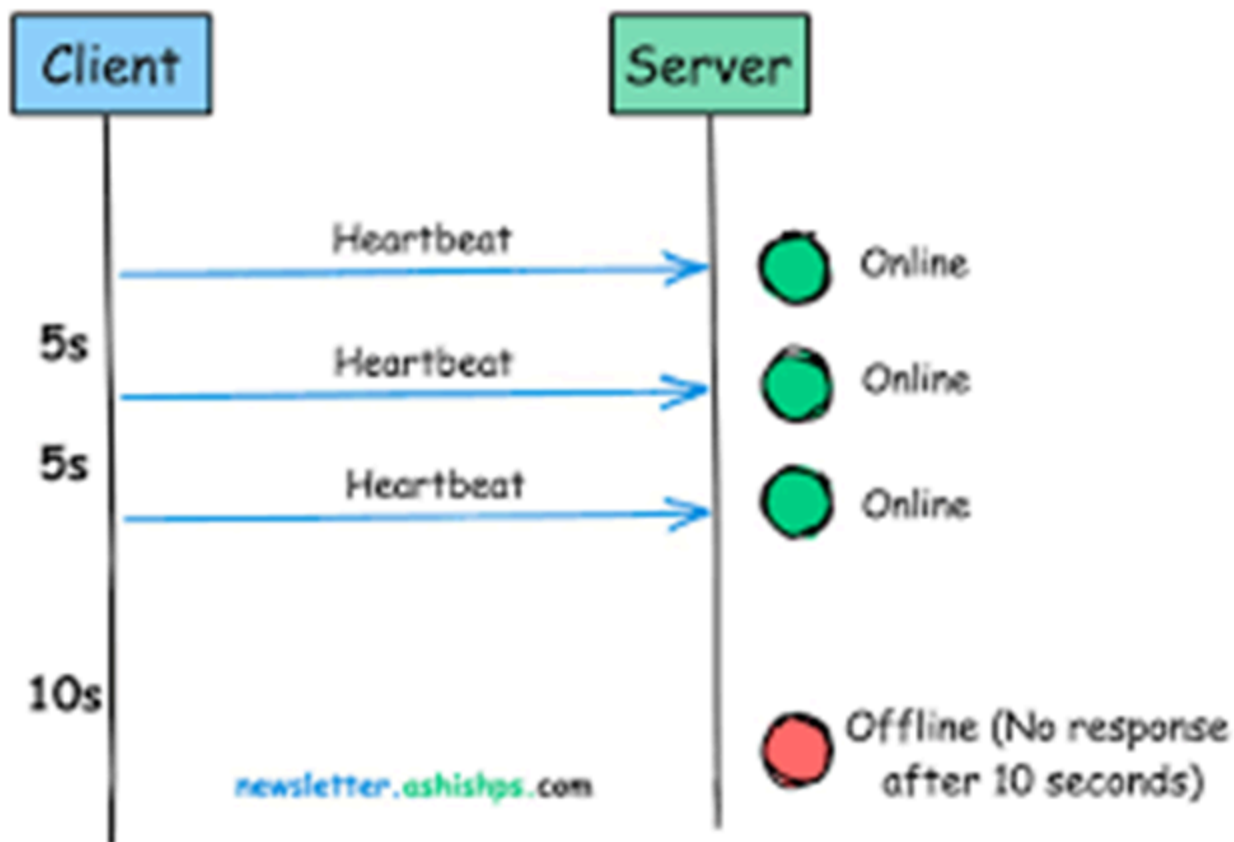
Heartbeat

A **heartbeat** is a periodic signal sent between system components to show they are alive and functioning correctly. If the heartbeat isn't received within a set timeout, the system assumes a failure and starts recovery actions.

Uses of Heartbeat

- Monitors the liveness of nodes, services, or applications
- Helps detect failures quickly when heartbeats stop

- Supports cluster operations like [failover](#), [leader election](#), and [service discovery](#)
Every 5 seconds → Node A sends a heartbeat to Node B.
If Node B misses 3 consecutive heartbeats → Node A is marked as failed.

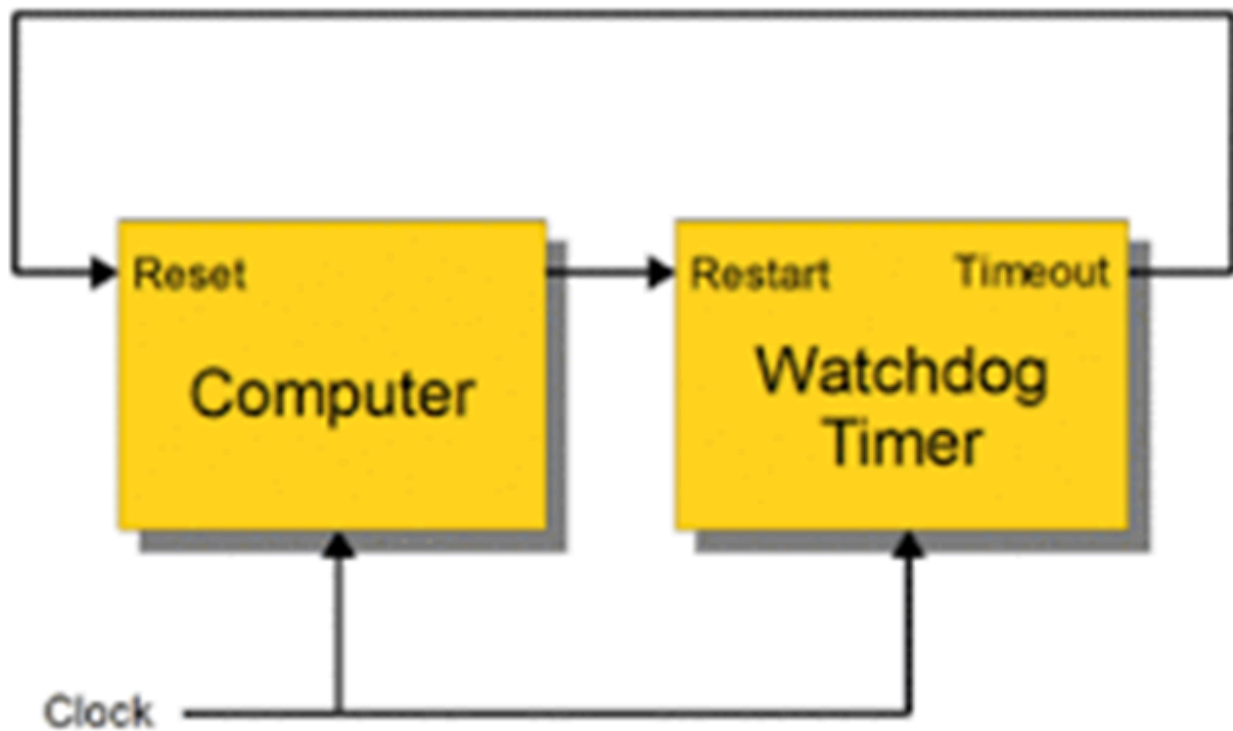


Watchdog Timers

A [watchdog timer](#) is a hardware or software timer that monitors a process and automatically resets or triggers corrective action if the process fails to respond within a defined time window. It prevents the system from getting stuck due to hangs or deadlocks.

Uses of Watchdog Timer

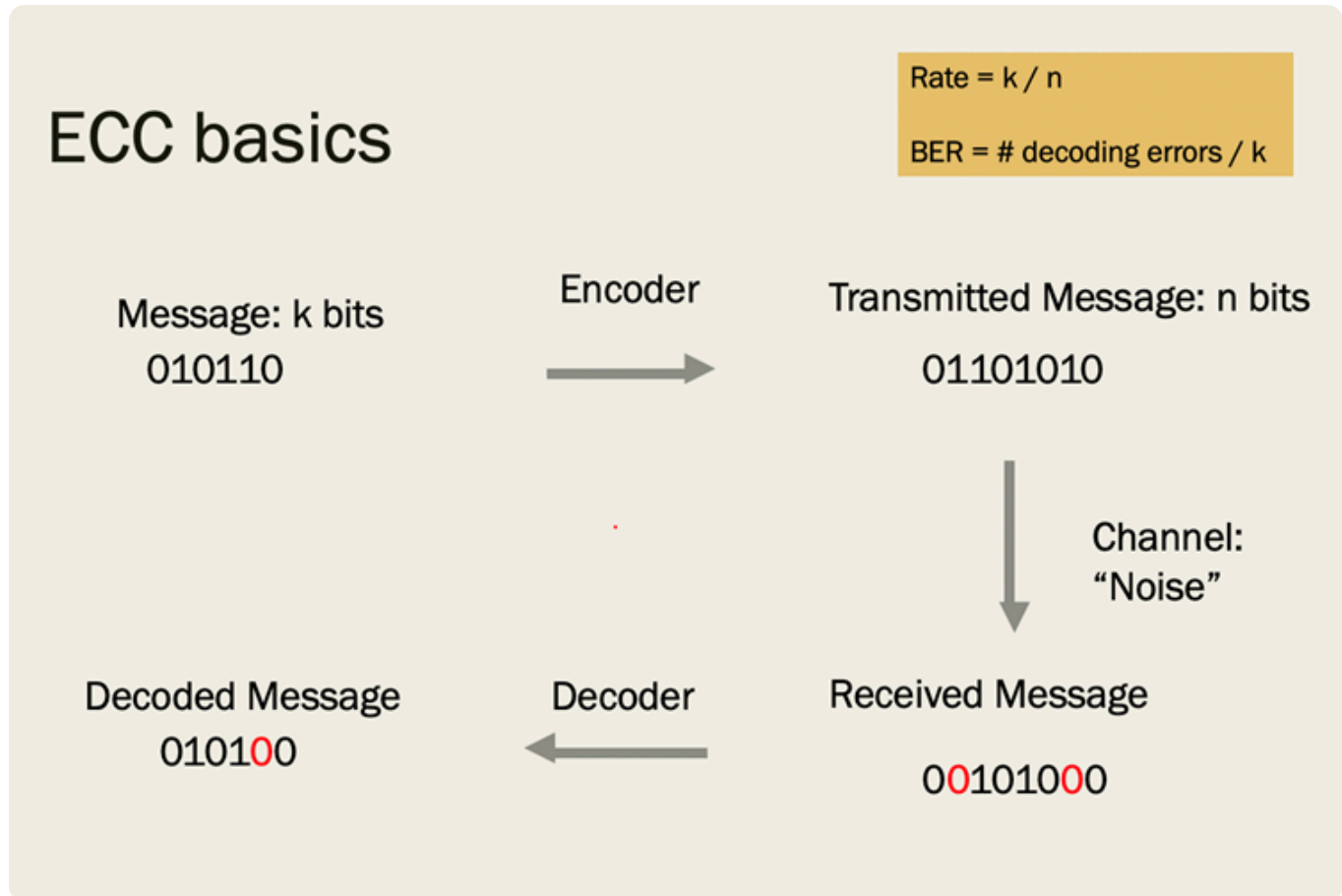
- Detects unresponsive behavior, hangs, or deadlocks
- Automatically restarts or resets faulty components
- Ensures reliability in embedded, real-time, and safety-critical systems



§

ECC (Error Correction Code):

ECC is a technique used to detect and correct data errors, mainly in memory and data transmission. It works by adding extra redundant bits to the original data, allowing the system to identify and fix errors caused by noise, interference, or hardware issues. ECC is essential for maintaining data integrity and reliability in critical systems like servers, storage devices, and

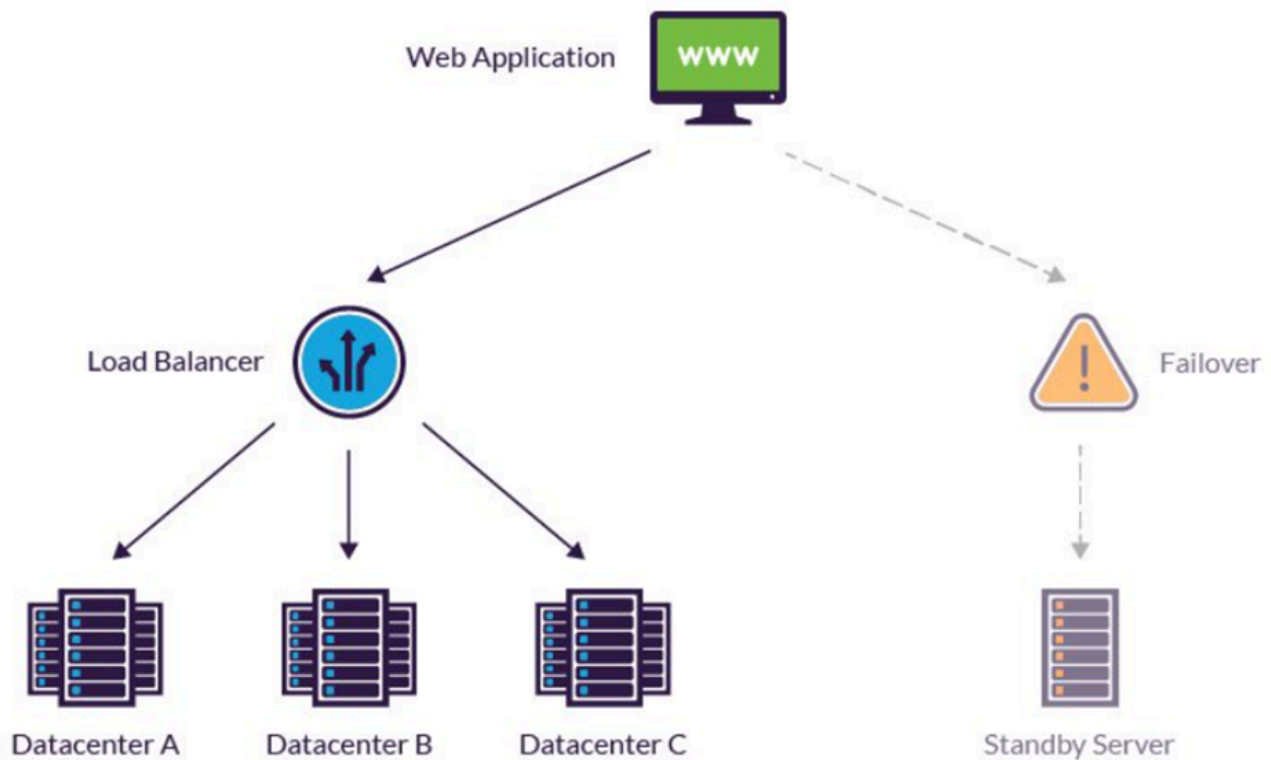


§

Fault Recovery

Fault recovery is the process of restoring normal system operation after a component fails, whether it's a server, network link, database, or application. It ensures the system continues functioning by shifting work to backup resources (**failover**) and later returning operations to the

original component once it's healthy again (**failback**).



↻

Failover

Failover is the mechanism of automatically transferring workloads, services, or data processing from a primary system to a backup system when a hardware failure, software crash, or planned maintenance occurs. It keeps critical services running without noticeable interruption to users. The backup system stays on standby, constantly synced with the primary, so it can take over instantly when needed.

Examples

- Switching from a primary database server to a replicated secondary server to keep transactions running.
- Triggering a VM failover to a backup VM hosted in another rack or data center.
- Routing network traffic through a standby router when the main router fails.

Phases of Failover

1. **Failure Detection**

The system notices missing heartbeats, errors, or downtime and flags the primary as failed.

2. **Initiating the Switch**

The cluster or controller activates the backup system, loads the last consistent state, and prepares it to take over operations.

3. **Redirecting Traffic**

User requests, data flows, or application traffic are rerouted to the backup system, ensuring continuity with minimal disruption.



Failback:

Failback is the process of shifting operations from the secondary (backup) system back to the original primary system once it has been repaired and verified as healthy. After a failover event, failback restores the system to its normal preferred configuration, making the primary take control again instead of the temporary backup.

Examples

- Syncing all updates made on the backup VM back to the original VM before switching control.
- Redirecting user or website traffic back to the primary server once it has been fully restored.
- Restoring a repaired storage array to its original role while the backup array returns to standby mode.

Phases of Failback

1. **Verify Primary System Health**

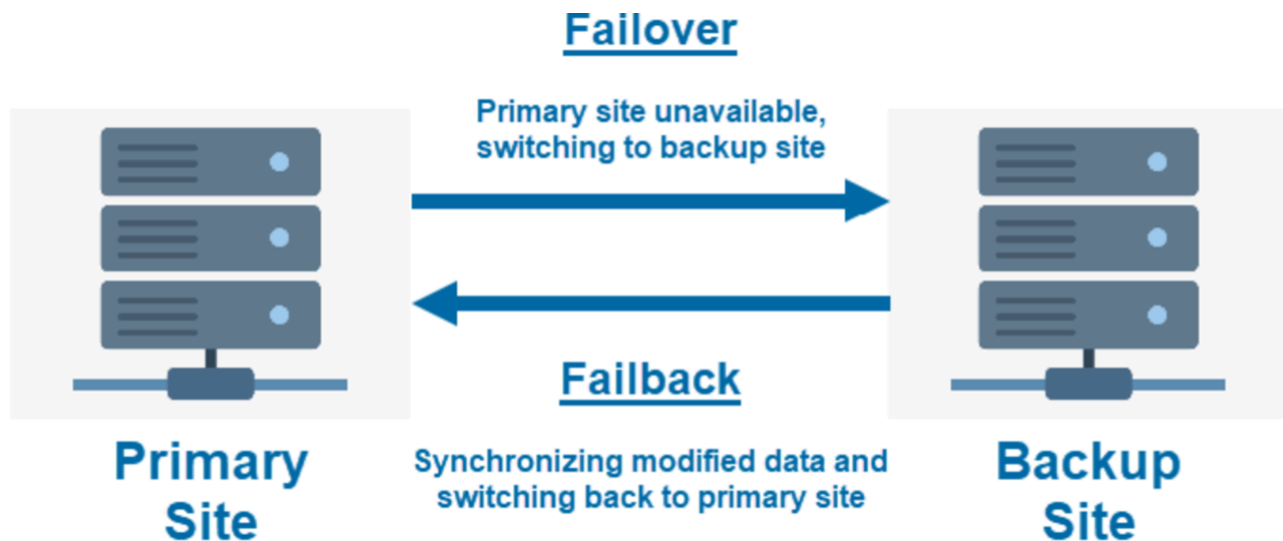
The system confirms the primary node is stable, fully repaired, and safe to take over operations again.

2. Synchronize Data Between Systems

Any changes that happened during failover are copied back to ensure the primary has updated, consistent data.

3. Redirect Traffic Back to Primary

Requests, workloads, and services are smoothly shifted from the backup system back to the primary without downtime.



Importance of Failover and Failback

- **Business Continuity:**

Failover and failback keep operations running during unexpected outages, forming the backbone of any disaster recovery strategy.

- **Reduced Downtime:**

Fast switching to backup systems during failover keeps downtime minimal, cutting losses and avoiding those angry “server down” emails.

- **Data Protection:**

Both processes rely on data replication, keeping information consistent and available across primary and backup systems.

- **Overall Purpose:**

Together, failover and failback deliver fault tolerance, quick recovery, and stable operations even after system failures.